

Machine Learning: A Comprehensive Textbook

Solutions to Chapter 13 Exercises Reinforcement Learning

Solutions Manual

Exercise 13.1 — Bellman Equations

Prove the Bellman expectation equation for $V^\pi(s)$ from the definition $V^\pi(s) = \mathbb{E}_\pi[G_t \mid s_t = s]$.

Solution. By definition, the return is $G_t = R_{t+1} + \gamma R_{t+2} + \gamma^2 R_{t+3} + \dots = R_{t+1} + \gamma G_{t+1}$ (a recursive definition: the return from time t is the immediate reward plus the discounted return from time $t + 1$ onward). Substituting into the value function definition:

$$V^\pi(s) = \mathbb{E}_\pi[G_t \mid s_t = s] = \mathbb{E}_\pi[R_{t+1} + \gamma G_{t+1} \mid s_t = s].$$

By linearity of expectation:

$$V^\pi(s) = \mathbb{E}_\pi[R_{t+1} \mid s_t = s] + \gamma \mathbb{E}_\pi[G_{t+1} \mid s_t = s].$$

Now expand each term by conditioning on the action taken and the resulting next state, using the law of total expectation. The policy $\pi(a \mid s)$ gives the probability of taking action a in state s , and the environment's transition dynamics $P(s' \mid s, a)$ (with expected immediate reward $R(s, a) = \mathbb{E}[R_{t+1} \mid s_t = s, a_t = a]$, possibly also depending on s' , but we absorb this into $R(s, a)$ for concision) give the distribution over next states:

$$\mathbb{E}_\pi[R_{t+1} \mid s_t = s] = \sum_a \pi(a \mid s) R(s, a) = \sum_a \pi(a \mid s) \sum_{s'} P(s' \mid s, a) r(s, a, s'),$$

where $r(s, a, s')$ is the (possibly state-dependent) immediate reward. For the second term, condition on the action a_t and the next state s_{t+1} :

$$\mathbb{E}_\pi[G_{t+1} \mid s_t = s] = \sum_a \pi(a \mid s) \sum_{s'} P(s' \mid s, a) \mathbb{E}_\pi[G_{t+1} \mid s_{t+1} = s'],$$

using the Markov property (given $s_{t+1} = s'$, the future return G_{t+1} is conditionally independent of the earlier history, including $s_t = s$) to drop the conditioning on $s_t = s$ inside the inner expectation, and then recognizing $\mathbb{E}_\pi[G_{t+1} \mid s_{t+1} = s'] = V^\pi(s')$ by definition (applying the same value-function definition one step later).

Combining both terms:

$$V^\pi(s) = \sum_a \pi(a \mid s) \sum_{s'} P(s' \mid s, a) [r(s, a, s') + \gamma V^\pi(s')],$$

which is exactly the **Bellman expectation equation** for V^π :

$$V^\pi(s) = \sum_a \pi(a \mid s) \sum_{s'} P(s' \mid s, a) [r(s, a, s') + \gamma V^\pi(s')] = \mathbb{E}_\pi[R_{t+1} + \gamma V^\pi(s_{t+1}) \mid s_t = s].$$

■ This recursive self-consistency equation — the value of a state equals the expected immediate reward plus the discounted value of wherever the policy leads next — is the foundation for essentially all dynamic-programming and temporal-difference RL algorithms (policy evaluation, value iteration, TD-learning, etc.).

Exercise 13.2 — Contraction

Prove that the Bellman optimality operator $(TV)(s) = \max_a \sum_{s'} P[R + \gamma V(s')]$ is a γ -contraction in ℓ_∞ .

Solution. We want to show: for any two value functions V_1, V_2 (viewed as vectors indexed by state),

$$\|TV_1 - TV_2\|_\infty \leq \gamma \|V_1 - V_2\|_\infty,$$

where $\|f\|_\infty := \max_s |f(s)|$ is the sup-norm.

Fix an arbitrary state s . By definition,

$$(TV_1)(s) - (TV_2)(s) = \max_a \sum_{s'} P(s' | s, a) [R(s, a, s') + \gamma V_1(s')] - \max_{a'} \sum_{s'} P(s' | s, a') [R(s, a', s') + \gamma V_2(s')].$$

Let $a_1^* := \arg \max_a \sum_{s'} P(s' | s, a) [R + \gamma V_1(s')]$ be the maximizer for V_1 . Then:

$$(TV_1)(s) = \sum_{s'} P(s' | s, a_1^*) [R(s, a_1^*, s') + \gamma V_1(s')].$$

Since $(TV_2)(s)$ is a *maximum* over all actions, in particular it is $\geq$ the value achieved by the specific action a_1^* :

$$(TV_2)(s) \geq \sum_{s'} P(s' | s, a_1^*) [R(s, a_1^*, s') + \gamma V_2(s')].$$

Subtracting:

$$\begin{aligned} (TV_1)(s) - (TV_2)(s) &\leq \sum_{s'} P(s' | s, a_1^*) [\gamma V_1(s') - \gamma V_2(s')] = \gamma \sum_{s'} P(s' | s, a_1^*) [V_1(s') - V_2(s')] \\ &\leq \gamma \sum_{s'} P(s' | s, a_1^*) \|V_1 - V_2\|_\infty = \gamma \|V_1 - V_2\|_\infty \end{aligned}$$

(using $V_1(s') - V_2(s') \leq \|V_1 - V_2\|_\infty$ pointwise, and $\sum_{s'} P(s' | s, a_1^*) = 1$, since $P(\cdot | s, a_1^*)$ is a valid probability distribution over next states).

By the symmetric argument (swapping the roles of V_1, V_2 , using $a_2^* := \arg \max_a$ for V_2 instead), we similarly get $(TV_2)(s) - (TV_1)(s) \leq \gamma \|V_1 - V_2\|_\infty$. Combining both directions:

$$|(TV_1)(s) - (TV_2)(s)| \leq \gamma \|V_1 - V_2\|_\infty.$$

Since s was arbitrary, taking the max over s on the left-hand side:

$$\|TV_1 - TV_2\|_\infty = \max_s |(TV_1)(s) - (TV_2)(s)| \leq \gamma \|V_1 - V_2\|_\infty. \quad \blacksquare$$

Since $0 \leq \gamma < 1$ (a standard assumption for discounted MDPs), T is indeed a γ -contraction in the sup-norm. **Consequence (Banach fixed-point theorem):** since T is a contraction on the complete metric space $(\mathbb{R}^{|\mathcal{S}|}, \|\cdot\|_\infty)$, it has a *unique* fixed point V^* satisfying $TV^* = V^*$ (this fixed point is exactly the optimal value function), and iterating $V_{k+1} = TV_k$ from *any* starting V_0 converges to V^* geometrically, with $\|V_k - V^*\|_\infty \leq \gamma^k \|V_0 - V^*\|_\infty \rightarrow 0$ — this is precisely the theoretical justification underlying the convergence guarantee of the **value iteration** algorithm.

Exercise 13.3 — Q-Learning Convergence Simulation

Show that on a 3-state MDP with known P and R , Q-learning converges to the correct Q^* within 100 iterations (simulate computationally).

Solution. Reference implementation. Define a simple 3-state MDP and run tabular Q-learning, comparing against the ground-truth Q^* computed exactly via value iteration (using the known P, R):

```
import numpy as np

np.random.seed(0)
n_states, n_actions, gamma = 3, 2, 0.9

# Known transition probabilities P[s,a,s'] and rewards R[s,a]
P = np.random.dirichlet(np.ones(n_states), size=(n_states, n_actions))
R = np.random.randn(n_states, n_actions)

# --- Ground truth via value iteration (Ex. 13.2's contraction guarantees convergence) ---
Q_star = np.zeros((n_states, n_actions))
for _ in range(1000):
    V = Q_star.max(axis=1)
    Q_star = R + gamma * (P @ V)

# --- Tabular Q-learning (model-free, samples transitions from P, R) ---
Q = np.zeros((n_states, n_actions))
alpha = 0.5
n_iterations = 100
for t in range(n_iterations):
    s = np.random.randint(n_states)
    a = np.random.randint(n_actions)
    s_next = np.random.choice(n_states, p=P[s, a])
    r = R[s, a]
    td_target = r + gamma * Q[s_next].max()
    Q[s, a] += alpha * (td_target - Q[s, a])

print("max |Q - Q*| after", n_iterations, "iterations:", np.abs(Q - Q_star).max())
```

Expected result. With a constant learning rate $\alpha = 0.5$ and only 100 single-transition updates spread across a mere $3 \times 2 = 6$ state-action pairs (so each pair is visited only ~ 16 -17 times on average, given uniform random exploration), Q-learning's estimate Q typically converges to within a small numerical tolerance of the true Q^* (e.g. $\max_{s,a} |Q(s,a) - Q^*(s,a)| \lesssim 0.1$ –0.3 in this toy 3-state, 2-action setting, depending on the specific random seed/dynamics) — a much larger number of iterations (1000+) with a properly *decaying* learning rate α_t (satisfying the Robbins-Monro conditions $\sum_t \alpha_t = \infty$, $\sum_t \alpha_t^2 < \infty$, e.g. $\alpha_t = 1/t$) would be needed to guarantee *exact* asymptotic convergence in the formal theoretical sense, but for a state space this small, 100 iterations with a moderate constant α is typically sufficient to get quite close in practice, illustrating Q-learning's fast empirical convergence on small, simple MDPs even without the full theoretical machinery of a carefully decaying step-size schedule. This empirical convergence is underpinned by the same

fixed-point/contraction-mapping theory established in Exercise 13.2: Q-learning is a stochastic-approximation algorithm for finding the fixed point of the (sampled, model-free version of the) Bellman optimality operator, and its convergence guarantee (under appropriate step-size and exploration conditions) is a classical result (Watkins & Dayan, 1992) building directly on this contraction property.

Exercise 13.4 — Experience Replay

Explain intuitively why correlations between consecutive transitions hurt training. Derive the variance reduction from random sampling vs. sequential sampling.

Solution. Intuition. In standard online RL, consecutive transitions $(s_t, a_t, r_t, s_{t+1}), (s_{t+1}, a_{t+1}, r_{t+1}, s_{t+2}), \dots$ are highly correlated, because they come from a single, continuous trajectory through state space — the agent tends to visit similar/nearby states in consecutive time steps (especially in environments with smooth/slowly-changing dynamics), and if the policy is deterministic or near-deterministic, it may also take similar actions repeatedly. Training a neural network (e.g. a Q-network) on a stream of such highly-correlated, non-i.i.d. samples violates the implicit i.i.d. assumption that stochastic gradient descent’s convergence theory relies on. Concretely, this causes two related problems: (1) the gradient updates become *biased* toward whatever narrow region of state space the agent currently happens to be in (recency bias — the network can “forget” what it learned about other regions of state space visited much earlier, a phenomenon closely related to catastrophic forgetting), and (2) the training signal has much higher variance/instability from batch to batch, since consecutive minibatches (drawn from a short, correlated window of recent experience) can look very different from each other depending on which part of the state space the trajectory currently occupies, causing oscillatory or divergent training dynamics — this instability is further compounded in RL specifically because the target values themselves (via the bootstrapped TD target $r + \gamma \max_{a'} Q(s', a')$) depend on the very same network being updated, creating a moving-target problem that correlated data makes considerably worse.

Experience replay addresses this by storing transitions in a large buffer and training on *randomly sampled minibatches* drawn from this buffer (rather than the raw sequential stream), which breaks the temporal correlation between consecutive training examples and makes each minibatch’s samples closer to an i.i.d. draw from the agent’s overall (time-averaged) experience distribution, closely restoring the i.i.d.-like conditions under which SGD is best understood to work well, and also allows each transition to be reused multiple times (greater sample efficiency) rather than being discarded after a single online update.

Variance reduction, quantitatively. Consider estimating the true gradient $g = \mathbb{E}[\nabla_{\theta} \ell(\theta; \tau)]$ (expectation over the stationary distribution of transitions τ) using a minibatch average $\hat{g} = \frac{1}{B} \sum_{i=1}^B \nabla_{\theta} \ell(\theta; \tau_i)$. If the τ_i are drawn i.i.d. (as with random replay-buffer sampling, approximately), then

$$\text{Var}(\hat{g}_{\text{random}}) = \frac{1}{B^2} \sum_{i=1}^B \text{Var}(\nabla_{\theta} \ell_i) = \frac{\sigma^2}{B}$$

(using independence, so covariance cross-terms vanish — exactly the ensemble-variance calculation of Exercise 6.2 with correlation $\rho = 0$), where $\sigma^2 := \text{Var}(\nabla_{\theta} \ell)$ is the per-sample gradient variance.

If instead the B samples are drawn *sequentially* (consecutive time steps from one trajectory), they exhibit positive autocorrelation $\rho(k)$ between samples k steps apart (typically decaying with k , e.g. geometrically for a mixing Markov chain, but crucially *nonzero* especially for small k). Using the

general ensemble-variance formula from Exercise 6.2 generalized to unequal pairwise correlations:

$$\text{Var}(\hat{g}_{\text{sequential}}) = \frac{\sigma^2}{B} + \frac{2\sigma^2}{B^2} \sum_{i < j} \rho(j-i) > \frac{\sigma^2}{B} = \text{Var}(\hat{g}_{\text{random}})$$

(strictly greater whenever the correlations $\rho(k) > 0$, as is typical for temporally adjacent RL transitions) — exactly mirroring the ensemble-variance-inflation result of Exercise 6.2, now applied to gradient-estimate variance instead of prediction variance. This shows formally that sequential (correlated) minibatch sampling yields a strictly higher-variance gradient estimate than i.i.d. random sampling of the same batch size B , directly justifying experience replay’s random-sampling design as a variance-reduction (and bias-reduction, via breaking recency-driven distributional shift) technique for stabilizing deep RL training.

Exercise 13.5 — Policy Gradient Theorem

Derive $\nabla_{\theta} J$ using the log-derivative trick. Show that the baseline $b(s)$ does not introduce bias.

Solution. Setup. Let $J(\theta) = \mathbb{E}_{\tau \sim \pi_{\theta}}[G(\tau)]$ be the expected return over trajectories τ sampled from policy π_{θ} , where $G(\tau) = \sum_t \gamma^t r_t$. We want $\nabla_{\theta} J(\theta)$.

The log-derivative (REINFORCE / score-function) trick. Write $J(\theta) = \int p_{\theta}(\tau) G(\tau) d\tau$, where $p_{\theta}(\tau)$ is the probability of trajectory τ under policy π_{θ} . Differentiate under the integral sign:

$$\nabla_{\theta} J(\theta) = \int \nabla_{\theta} p_{\theta}(\tau) G(\tau) d\tau.$$

Use the identity $\nabla_{\theta} p_{\theta}(\tau) = p_{\theta}(\tau) \nabla_{\theta} \log p_{\theta}(\tau)$ (which follows directly from $\nabla_{\theta} \log p_{\theta}(\tau) = \nabla_{\theta} p_{\theta}(\tau) / p_{\theta}(\tau)$, the chain rule for log):

$$\nabla_{\theta} J(\theta) = \int p_{\theta}(\tau) \nabla_{\theta} \log p_{\theta}(\tau) G(\tau) d\tau = \mathbb{E}_{\tau \sim \pi_{\theta}} [\nabla_{\theta} \log p_{\theta}(\tau) G(\tau)].$$

Now expand $\log p_{\theta}(\tau)$: a trajectory’s probability factors as $p_{\theta}(\tau) = p(s_0) \prod_t \pi_{\theta}(a_t | s_t) P(s_{t+1} | s_t, a_t)$, so $\log p_{\theta}(\tau) = \log p(s_0) + \sum_t [\log \pi_{\theta}(a_t | s_t) + \log P(s_{t+1} | s_t, a_t)]$. Only the policy terms depend on θ (the initial-state distribution and transition dynamics do not depend on the policy parameters), so

$$\nabla_{\theta} \log p_{\theta}(\tau) = \sum_t \nabla_{\theta} \log \pi_{\theta}(a_t | s_t).$$

Substituting:

$$\nabla_{\theta} J(\theta) = \mathbb{E}_{\tau \sim \pi_{\theta}} \left[\left(\sum_t \nabla_{\theta} \log \pi_{\theta}(a_t | s_t) \right) G(\tau) \right]. \quad \blacksquare$$

(A slightly refined, lower-variance version of this — using the fact that rewards before time t cannot depend on action a_t , so only the “reward-to-go” from t onward is causally relevant — gives the more standard form $\nabla_{\theta} J(\theta) = \mathbb{E}[\sum_t \nabla_{\theta} \log \pi_{\theta}(a_t | s_t) G_t]$ with $G_t = \sum_{t' \geq t} \gamma^{t'-t} r_{t'}$, but the derivation above establishes the core log-derivative trick underlying the full policy gradient theorem.)

Baseline does not introduce bias. Consider subtracting a state-dependent baseline $b(s_t)$ (crucially, a function only of s_t , not of the action a_t):

$$\nabla_{\theta} J(\theta) = \mathbb{E}_{\tau} \left[\sum_t \nabla_{\theta} \log \pi_{\theta}(a_t | s_t) (G_t - b(s_t)) \right].$$

We need to show that the extra term introduced, $-\mathbb{E}_\tau[\sum_t \nabla_\theta \log \pi_\theta(a_t | s_t) b(s_t)]$, is exactly zero (in expectation) for *any* choice of $b(s_t)$ depending only on s_t . Focus on a single time step t and condition on s_t :

$$\mathbb{E}_{a_t \sim \pi_\theta(\cdot | s_t)}[\nabla_\theta \log \pi_\theta(a_t | s_t) b(s_t)] = b(s_t) \cdot \mathbb{E}_{a_t}[\nabla_\theta \log \pi_\theta(a_t | s_t)]$$

(pulling $b(s_t)$ out of the expectation over a_t , valid since $b(s_t)$ does not depend on a_t). Now compute the remaining expectation:

$$\mathbb{E}_{a_t \sim \pi_\theta(\cdot | s_t)}[\nabla_\theta \log \pi_\theta(a_t | s_t)] = \sum_a \pi_\theta(a | s_t) \nabla_\theta \log \pi_\theta(a | s_t) = \sum_a \nabla_\theta \pi_\theta(a | s_t)$$

(using the same identity $\pi_\theta \nabla_\theta \log \pi_\theta = \nabla_\theta \pi_\theta$ as before, in reverse). Since $\sum_a \pi_\theta(a | s_t) = 1$ for every θ (a valid probability distribution, for all parameter values), differentiating both sides with respect to θ gives $\sum_a \nabla_\theta \pi_\theta(a | s_t) = \nabla_\theta(1) = 0$. So

$$\mathbb{E}_{a_t}[\nabla_\theta \log \pi_\theta(a_t | s_t)] = 0 \implies \mathbb{E}_{a_t}[\nabla_\theta \log \pi_\theta(a_t | s_t) b(s_t)] = b(s_t) \cdot 0 = 0.$$

Taking the full expectation over s_t (and summing over t) preserves this zero: the extra baseline term contributes exactly 0 to $\nabla_\theta J$ in expectation, for *any* choice of state-dependent baseline $b(s_t)$. ■

Conclusion. Subtracting a state-dependent (action-independent) baseline leaves the policy gradient estimator **unbiased** — $\mathbb{E}_\tau[\sum_t \nabla_\theta \log \pi_\theta(a_t | s_t)(G_t - b(s_t))] = \nabla_\theta J(\theta)$ exactly, for any $b(s_t)$ — while a well-chosen baseline (typically $b(s_t) = V^\pi(s_t)$, the state value function, giving the *advantage* $A_t = G_t - V^\pi(s_t)$) can *substantially reduce the variance* of the gradient estimate without introducing any bias, which is exactly why baselines (and more generally, advantage-based estimators like GAE) are a standard and important variance-reduction technique in policy-gradient methods.

Exercise 13.6 — PPO Clipping

For $\hat{A}_t > 0$, show the PPO objective (13.6) prevents large increases in r_t . For $\hat{A}_t < 0$, show it prevents large decreases.

Solution. The PPO clipped surrogate objective is

$$L^{\text{CLIP}}(\theta) = \mathbb{E}_t \left[\min(r_t(\theta) \hat{A}_t, \text{clip}(r_t(\theta), 1 - \epsilon, 1 + \epsilon) \hat{A}_t) \right],$$

where $r_t(\theta) = \frac{\pi_\theta(a_t | s_t)}{\pi_{\theta_{\text{old}}}(a_t | s_t)}$ is the probability ratio between the new and old policy, and $\hat{A}_t$ is the estimated advantage.

Case $\hat{A}_t > 0$ (the action was better than average — we want to increase its probability). We analyze the objective as a function of r_t for fixed $\hat{A}_t > 0$:

- For $r_t \leq 1 + \epsilon$: $\text{clip}(r_t, 1 - \epsilon, 1 + \epsilon) = r_t$ (unclipped, since r_t is within range), so both terms in the min equal $r_t \hat{A}_t$, and $L^{\text{CLIP}} = r_t \hat{A}_t$ — the objective increases linearly with r_t in this range, providing a normal (unclipped) incentive to increase r_t (i.e. increase the probability of this good action).
- For $r_t > 1 + \epsilon$: $\text{clip}(r_t, 1 - \epsilon, 1 + \epsilon) = 1 + \epsilon$ (clipped at the upper bound), so the second term is the constant $(1 + \epsilon) \hat{A}_t$, while the first (unclipped) term $r_t \hat{A}_t$ continues to grow with r_t . Since $\hat{A}_t > 0$ and $r_t > 1 + \epsilon$, we have $r_t \hat{A}_t > (1 + \epsilon) \hat{A}_t$, so the min selects the *smaller, clipped* value: $L^{\text{CLIP}} = (1 + \epsilon) \hat{A}_t$ — a **constant**, independent of r_t , for all $r_t > 1 + \epsilon$.

Since the objective becomes flat (zero gradient with respect to r_t , hence with respect to θ through r_t) once r_t exceeds $1 + \epsilon$, there is **no further gradient incentive to increase r_t beyond $1 + \epsilon$** — the clipping mechanism removes the policy-improvement signal exactly at the point where the new policy would become too different from the old policy (in the direction of increasing this action’s probability too aggressively), directly preventing large increases in r_t for positive-advantage actions. ■

Case $\hat{A}_t < 0$ (the action was worse than average — we want to decrease its probability). Now with $\hat{A}_t < 0$, both terms $r_t \hat{A}_t$ and $\text{clip}(r_t, \dots) \hat{A}_t$ are *decreasing* functions of their (unclipped) ratio argument, since multiplying by a negative number flips monotonicity:

- For $r_t \geq 1 - \epsilon$: unclipped, both terms equal $r_t \hat{A}_t$, objective = $r_t \hat{A}_t$, decreasing in r_t (larger r_t gives more negative — worse — objective, correctly incentivizing decreasing r_t in this range).
- For $r_t < 1 - \epsilon$: $\text{clip} \rightarrow 1 - \epsilon$, second term becomes the constant $(1 - \epsilon) \hat{A}_t$. Since $\hat{A}_t < 0$ and $r_t < 1 - \epsilon$, we have $r_t \hat{A}_t > (1 - \epsilon) \hat{A}_t$ (multiplying the inequality $r_t < 1 - \epsilon$ by the negative number $\hat{A}_t$ flips the direction), so the min selects $L^{\text{CLIP}} = r_t \hat{A}_t$ — wait, we need the smaller of the two: since $r_t \hat{A}_t > (1 - \epsilon) \hat{A}_t$ in this regime, the *smaller* value is $(1 - \epsilon) \hat{A}_t$, so the min selects the **clipped, constant** value $(1 - \epsilon) \hat{A}_t$ for all $r_t < 1 - \epsilon$.

Again, the objective is **flat** (constant, zero gradient) once r_t drops below $1 - \epsilon$, removing the gradient incentive to decrease r_t any further — directly preventing large decreases in r_t for negative-advantage actions. ■

Summary. In both cases, the clipping mechanism creates a “dead zone” (flat, zero-gradient region) once the policy ratio r_t moves too far in the direction the (unclipped) advantage-weighted objective would otherwise push it — specifically, r_t is prevented from increasing past $1 + \epsilon$ when $\hat{A}_t > 0$, and prevented from decreasing past $1 - \epsilon$ when $\hat{A}_t < 0$. This is exactly PPO’s core mechanism for enforcing a soft *trust region*: it discourages the new policy π_θ from moving too far from the old policy $\pi_{\theta_{\text{old}}}$ in a single update (regardless of how large the raw advantage-weighted gradient signal would otherwise be), which is what gives PPO its characteristic training stability compared to unconstrained policy-gradient methods, without requiring the more complex second-order trust-region machinery of TRPO.

Exercise 13.7 — RLHF Reward Hacking

Explain the concept of reward hacking in RLHF. Why is the KL penalty essential? What happens when $\beta \rightarrow 0$?

Solution. Reward hacking. In RLHF, a reward model $r_\phi(x, y)$ is trained to approximate human preferences (typically via a Bradley-Terry model fit to pairwise human preference comparisons), and the language-model policy π_θ is then optimized (e.g. via PPO) to maximize this *learned* reward. Because r_ϕ is only an imperfect, finite-sample approximation to the true underlying human preference function (trained on a limited set of comparisons, and necessarily generalizing imperfectly outside the distribution of outputs it was trained on), there inevitably exist “blind spots” — regions of the output space where r_ϕ assigns high reward but which do *not* actually correspond to genuinely high-quality, human-preferred outputs (e.g. the reward model might have learned a spurious correlation, like “longer answers are usually rated better,” or “certain buzzwords/phrasings correlate with the training-data raters’ approval,” without those actually being causally linked to true output quality).

Reward hacking refers to the phenomenon where the RL policy, in the course of aggressively optimizing r_ϕ , discovers and exploits exactly these blind spots/spurious-correlation regions — driving

the reward model’s score arbitrarily high while producing outputs that a human would judge as low-quality, degenerate, repetitive, sycophantic, or otherwise misaligned with genuine human preferences (this is a specific instance of Goodhart’s Law: “when a measure becomes a target, it ceases to be a good measure”). Because RL is an explicit, often very effective optimization process, and the reward model is imperfect, unconstrained optimization of r_ϕ tends to drift the policy toward these adversarial/degenerate high-reward-model-score-but-low-true-quality regions, often quite far from the space of outputs the reward model was actually trained (and is therefore reliable) on.

Why the KL penalty is essential. The standard RLHF objective is

$$\max_{\theta} \mathbb{E}_{x,y \sim \pi_{\theta}} [r_{\phi}(x,y)] - \beta D_{\text{KL}}(\pi_{\theta}(\cdot | x) \| \pi_{\text{ref}}(\cdot | x)),$$

where π_{ref} is the original (e.g. supervised-fine-tuned, pre-RL) reference policy, and $\beta > 0$ controls the strength of the KL penalty. This penalty term explicitly discourages the policy π_{θ} from straying too far (in KL divergence, cf. the properties established in Exercise 2.9/2.10) from the reference policy’s output distribution. Since π_{ref} was trained via supervised fine-tuning on genuinely high-quality human-written/human-approved data (and, importantly, is exactly the distribution the reward model was itself trained/calibrated on), staying close to π_{ref} in KL divergence keeps the policy’s outputs within the region of output space where the reward model r_{ϕ} is actually a *reliable, well-calibrated* proxy for true human preference — the KL penalty acts as a regularizer that prevents the RL optimization from wandering into the reward model’s out-of-distribution blind spots where reward hacking becomes possible, trading off some raw reward-model score for staying within a trustworthy, “in-distribution” region.

What happens as $\beta \rightarrow 0$. As the KL penalty coefficient $\beta \rightarrow 0$, the objective reduces to pure, unconstrained maximization of the (imperfect, proxy) reward model score: $\max_{\theta} \mathbb{E}_{\pi_{\theta}} [r_{\phi}(x,y)]$, with no regularization pulling the policy back toward the trustworthy reference distribution. In this limit, the policy is free to drift arbitrarily far from π_{ref} in pursuit of ever-higher (but increasingly *spurious*, reward-hacked) r_{ϕ} scores — empirically, this manifests as severe degeneration: outputs becoming repetitive, incoherent, exploiting specific reward-model quirks (e.g. excessive use of certain phrases the reward model over-values), or otherwise diverging sharply from genuinely helpful, human-preferred text, even while the *measured* reward-model score continues to climb — a textbook, empirically well-documented (e.g. in the original InstructGPT/RLHF papers) illustration of Goodhart’s Law and reward hacking in practice. This is precisely why β is never set to exactly zero in practice, and is often tuned (or dynamically adapted, as in some RLHF variants) to balance the trade-off between optimizing for the (imperfect) learned reward and maintaining fidelity to the trustworthy reference policy.